Department of Computer Science & Engineering
The University of Texas at Arlington

System Requirements Specification
CSE 4316: Senior Design I
Fall 2025

Team Name
Product Name

Preston Loera
Matthew Nguyen
Marshall Helton
David Daffern
Aadhitya Kumar
Tung Nguyen

Revision History

Revision	Date	Author(s)	Description
0.1	10.15.2025	MH, DD	document creation
0.2	10.17.2025	MH, DD	complete draft, added AI-assisted content revisions (ChatGPT, reviewed by team)
0.3	04.08.2026	MH, DD	Update descriptions based on BASIL Ladder submission and slight scope changes

Contents

1 Product Concept **6**
1.1 Purpose and Use . . . 6
1.2 Intended Audience . . . 7

2 Product Description **8**
2.1 Features & Functions . . . 8
2.2 External Inputs & Outputs . . . 8
2.3 Product Interfaces . . . 9

3 Customer Requirements **11**
3.1 CR-1 Autonomous Match Play . . . 11
3.2 CR-2 Real-Time Latency Budget . . . 11
3.3 CR-3 Scouting & Opponent Modeling . . . 11
3.4 CR-4 Adaptive Strategy & Build Orders . . . 12
3.5 CR-5 Economy & Production Management . . . 12
3.6 CR-6 Combat Control (Squad-Level) . . . 13
3.7 CR-7 Logging, Metrics, & Replays . . . 13
3.8 CR-8 Configuration & Run Control . . . 13
3.9 CR-9 Compatibility & Environment . . . 14
3.10 CR-10 Robustness & Failure Handling . . . 14
3.11 CR-11 Developer Debug Overlays (Optional) . . . 14
3.12 CR-12 Ethical & Fair Play . . . 15
3.13 CR-13 Competitive Ladder Compatibility . . . 15

4 Packaging Requirements **16**
4.1 PR-1 Distribution Channel . . . 16
4.2 PR-2 Release Artifacts . . . 16
4.3 PR-3 Directory Layout . . . 16
4.4 PR-4 Setup & Launch Scripts . . . 17
4.5 PR-5 Configuration Package . . . 17
4.6 PR-6 Documentation Bundle . . . 18
4.7 PR-7 Compatibility Notes . . . 18
4.8 PR-8 Logging & Outputs Placement . . . 18
4.9 PR-9 Licensing & Attributions . . . 19
4.10 PR-10 Release Notes & Known Issues . . . 19
4.11 PR-11 Integrity & Provenance . . . 19
4.12 PR-12 Sample Assets for Verification . . . 20

5 Performance Requirements **21**
5.1 PRF-1 On-Frame Latency Budget . . . 21
5.2 PRF-2 Startup Time . . . 21
5.3 PRF-3 Shutdown & Artifact Flush . . . 21
5.4 PRF-4 Logging/Metric Throughput . . . 22
5.5 PRF-5 Memory Footprint . . . 22
5.6 PRF-6 CPU Utilization . . . 22
5.7 PRF-7 Pathfinding/Combat Budgeting . . . 23

5.8 PRF-8 Opponent Model Update Rate . . . 23
5.9 PRF-9 Config Load & Validation . . . 23
5.10 PRF-10 Replay & Metrics Size . . . 24
5.11 PRF-11 Frame Overrun Handling . . . 24
5.12 PRF-12 Measurement & Reporting . . . 24

6 Safety Requirements **26**
6.1 SR-1 Software Runtime Safety . . . 26
6.2 SR-2 Data and System Integrity Safety . . . 26
6.3 SR-3 Ethical and Competition Safety . . . 26

7 Maintenance & Support Requirements **28**
7.1 MSR-1 Source Code Maintenance and Version Control . . . 28
7.2 MSR-2 Configuration and Compatibility Maintenance . . . 28
7.3 MSR-3 Technical Support and Handoff . . . 28

8 Other Requirements **30**
8.1 OR-1 Modularity and Extensibility . . . 30
8.2 OR-2 wCoding Standards and Quality Assurance . . . 30
8.3 OR-3 Security and Data Privacy . . . 30

9 Future Items **32**
9.1 CR-11 Developer Debug Overlays (Optional) . . . 32
9.2 PR-11 Integrity & Provenance . . . 32
9.3 PR-12 Sample Assets for Verification . . . 32
9.4 OR-3 Security and Data Privacy . . . 33
9.5 MSR-4 Documentation and Support Materials . . . 33
9.6 MSR-5 Bug Tracking and Issue Resolution . . . 33

List of Figures

1 ProtoBot conceptual screenshot 7
2 High-level ProtoBot architecture showing managers, BWAPI integration, and data flows. 8

1 Product Concept

ProtoBot is an autonomous StarCraft game-playing agent built to make strong, adaptable decisions when information is incomplete. Instead of following rigid, all-in scripts, it synthesizes scouting cues and in-game telemetry to estimate hidden opponent state, adjust build orders on the fly, and coordinate economy, production, tech, and unit control in real time. The product is intended for AI researchers, competition organizers, and players who want a capable, explainable opponent and a repeatable test bed for strategy studies. Users run ProtoBot through the game's API to conduct full matches, evaluate strategic choices under fog-of-war, and benchmark performance against scripted and human opponents. By emphasizing learning-driven adaptation and transparent decision making, ProtoBot provides a practical platform for experimenting with intelligent play that goes beyond brittle rule sets.

1.1 Purpose and Use

ProtoBot is an autonomous StarCraft: Brood War agent that plays full 1v1 matches under fog-of-war. Its purpose is to demonstrate adaptive decision-making that goes beyond fixed, rule-only play by (a) estimating hidden enemy state from scouting, (b) adjusting strategy and build orders in real time, and (c) coordinating macro (economy/production/tech) with micro (unit control) within the tournament latency limit ($\leq$ 42 ms per frame).

ProtoBot is intended for AI research, competitive benchmarking, and player practice. Users run ProtoBot through BWAPI on StarCraft: Brood War 1.16.1 to execute full matches, gather performance metrics, and evaluate strategy modules under consistent conditions (fog-of-war enabled, fastest speed).

In scope (what it should do):

- Run end-to-end matches autonomously via BWAPI.
- Maintain an opponent model from scouting signals.
- Adapt build orders and strategy in real time.
- Manage economy/production to meet strategic goals.
- Control combat via squad-level abstractions ($\leq$ 42 ms/frame).
- Log match data for analysis and regression testing.

Primary usage scenarios:

- **Research and experimentation:** swap/ablate modules (e.g., scouting policy, strategy selector) and compare outcomes on fixed map sets.
- **Competition prep:** ProtoBot is also deployed in online competitive environments such as the BASIL Ladder, where it participates in automated matches against other bots. This enables continuous evaluation, ranking, and performance benchmarking under real tournament-like conditions.
- **Practice opponent:** provide consistent, explainable play for human players to test openings and counters.

Operational constraints and assumptions:

StarCraft: Brood War 1.16.1 with BWAPI; read/write access to standard bwapi-data folders.

Execution at setLocalSpeed(0) (fastest); $\leq$ 42 ms per frame on typical dev hardware.

Fog-of-war enabled; evaluation games capped by a fixed frame limit.

1.2 Intended Audience

ProtoBot is intended for users who need a reliable, explainable StarCraft: Brood War agent for research, evaluation, and practice under fog-of-war. The primary audience is AI practitioners who want to study decision making with strict latency constraints; secondary audiences include competition organizers and players who need a consistent sparring opponent. ProtoBot can be run as a standalone bot through BWAPI or embedded as a component within larger research pipelines (e.g., benchmarking harnesses, ablation studies).

Primary audiences

- *AI researchers and students* exploring planning under partial observability, opponent modeling, and real-time control with a ≤ 42 ms per-frame budget.
- *Competition organizers and participants in ladder environments* such as BASIL, who require an agent that adheres to common ladder/tournament constraints (BWAPI, fastest speed, fog-of-war enabled) for reproducible benchmarking.

Secondary audiences

- *Developers and hobbyists* seeking a modular reference bot (scouting, strategy, build, combat, worker, information managers) to extend or compare against.
- *Players and coaches* who want a stable, explainable practice opponent to test openings, counters, and macro/micro drills.

Purchase/use context

- *Public/academic use:* Open research and coursework, reproducible experiments, and teaching labs.
- *Commercial/organizational use:* Internal evaluation of RTS decision modules, bot-vs-bot testbeds, or integration as a component in analytics/visualization tools.

Scope of use

- *General use* as a standalone BWAPI bot for 1v1 Brood War 1.16.1 matches.
- *Component use* as a pluggable module within larger systems (e.g., swapping the strategy selector or scouting policy for ablation and benchmarking).

Figure 1: ProtoBot conceptual screenshot

2 Product Description

This section provides the reader with an overview of ProtoBot, an autonomous StarCraft: Brood War agent built to play full 1v1 matches under fog of war via BWAPI. From the perspectives of end users, maintainers, and administrators, we summarize how ProtoBot models hidden enemy state from scouting, adapts build orders and strategy in real time, manages economy/production, and controls combat within a strict $\leq$ 42 ms per-frame budget. The key features, user interactions, and visible interfaces are detailed below and mapped to the system's modular architecture (Information, Scouting, Strategy, Build, Worker, Combat, and Map Tools) referenced in Figure 2.

2.1 Features & Functions

What the product *does*.

- Plays full 1v1 matches of StarCraft: Brood War 1.16.1 autonomously via BWAPI with fog-of-war enabled.
- Maintains an opponent model using scouting observations and in-game telemetry to inform strategy selection.
- Selects and adapts build orders, expansion timing, and tech progression in real time.
- Manages economy and production (worker assignment, mineral/gas balance, supply planning).
- Controls combat at a squad level to satisfy the tournament budget of $\leq$ 42 ms per frame.
- Records match artifacts (logs, summaries, replays) for analysis, benchmarking, and regression testing.

What the product *does not* do (this release).

- Team formats (2v2, 3v3), custom non-ladder rules, or non-standard map mechanics.
- Heavy end-to-end training pipelines; ML is applied selectively where it fits the real-time constraint.

Principal components (see Figure 2).

- *Information Manager* — aggregates game state, scouting results, and belief over enemy tech/army.
- *Scouting Manager* — plans and executes scouting routes/policies; publishes observations.
- *Strategy Manager* — selects openings, transitions, and counters; updates build/production goals.
- *Build Manager* — turns strategic goals into build orders and structure/unit production queues.
- *Worker Manager* — allocates workers to minerals/gas, handles saturation and transfers.
- *Combat Manager* — abstracts units into squads; issues micro/macro combat actions.
- *Map Tools* — pathfinding and spatial utilities (influence maps, region/expansion analysis).
- *I/O Layer* — BWAPI interface, configuration loader, logging, replay and metrics emitters.

Figure 2: High-level ProtoBot architecture showing managers, BWAPI integration, and data flows.

2.2 External Inputs & Outputs

This section lists critical external data flows between ProtoBot, end users, and external systems (e.g., StarCraft client, filesystem). Tables 2 and 3 summarize inputs and outputs.

Table 2: External Inputs

Name	Description / Use
BWAPI Game State	Real-time observations: visibility, units, health, resources, tech; consumed by all managers.
Scouting Observations	Probe/worker/unit sightings, timings, locations; drive opponent model and counters.
Configuration File	Runtime options (race, opening set, logging level, output paths, feature flags).
Command-line Flags	Quick overrides for config (e.g., map pool, opponent, seed, debug overlays).
Map Files (.scx/.scm)	Standard Brood War maps; used by Map Tools for regions/expansions/pathfinding hints.
Opponent Binary	External bot or human opponent selection; establishes match-up and ladder rules.

Table 3: External Outputs

Name	Description / Use
In-Game Actions	Build/upgrade/unit micro/macro commands issued via BWAPI each frame.
Match Logs	Timestamped decisions, states, and events for debugging and analysis.
Metrics Summary	CSV/JSON of KPIs (win/loss, frame length, APM, army value, expansions, tech timings).
Replays (.rep)	Game replays for review, regression, and competition evidence.
Debug Overlays (opt.)	On-screen annotations (regions, squad intents) for developer builds.
Crash/Trace Reports	Error traces and last-known state to aid fault diagnosis.

2.3 Product Interfaces

End-user (player/coach) interface.

- Runs as a BWAPI bot; end users launch matches through the standard BWAPI launcher or scripts.
- Optional on-screen debug overlays (developer build) for visibility into scouting, region control, and squad plans.
- Replays and match summaries available post-game for study and practice planning.

Research/maintainer interface.

- *Configuration*: YAML/INI-style file plus command-line flags to enable/disable modules or set experiments.
- *Logging & metrics*: structured logs and CSV/JSON metrics to support ablations and benchmarks.
- *Artifacts*: automatic replay export and optional state snapshots to the standard `bwapi-data` folders.

Administrator/integration interface.

- Compatible with StarCraft: Brood War 1.16.1 and BWAPI; operates at `setLocalSpeed(0)` with fog-of-war on.
- Observes on-frame budget of ≤ 42 ms per frame; emits warnings if limits are exceeded.
- Read access to `bwapi-data/read/` and write access to `bwapi-data/write/` per ladder norms.

Ladder/Competition Interface

- ProtoBot supports execution within automated ladder environments (e.g., BASIL), where matches are scheduled and executed without user interaction. The system must integrate with standard BWAPI-based ladder infrastructure and adhere to match execution protocols.

3 Customer Requirements

This section specifies the customer-facing features and functions of ProtoBot. These requirements define the expected "look and feel," operational behaviors, and visible outcomes of the product for its intended users (AI researchers, competition organizers, and players). Each requirement states what ProtoBot must or should do from a user's perspective, along with source, constraints, applicable standards, and priority. Changes to these requirements must be coordinated with the project instructor and the team.

3.1 CR-1 Autonomous Match Play

Description

ProtoBot shall autonomously play full 1v1 StarCraft: Brood War 1.16.1 matches under fog-of-war via BWAPI with no human intervention during play (from game start to result), including execution within automated ladder environments.

Source

Instructor guidance; competition norms; team charter.

Constraints

- Game version and ladder rules; hardware performance limits.
- No external assistance or mid-match manual overrides.

Standards

- StarCraft: Brood War 1.16.1; BWAPI runtime conventions; standard ladder settings.

Priority

Critical

3.2 CR-2 Real-Time Latency Budget

Description

ProtoBot shall respect a per-frame computation budget of ≤ 42 ms; it shall avoid long stalls and surface warnings when limits are exceeded.

Source

Competition norms; team charter.

Constraints

- On-frame processing only; limited background precomputation.
- Performance variability across developer machines.

Standards

- Ladder timing policies; BWAPI callback timing behavior.

Priority

Critical

3.3 CR-3 Scouting & Opponent Modeling

Description

ProtoBot shall collect scouting observations and maintain a lightweight belief over enemy tech/army to inform strategic decisions.

Source

Research goals; team charter.

Constraints

- Partial observability (fog-of-war); noisy/incomplete signals.
- Must remain within the latency budget.

Standards

- Internal data schemas for observations; replay semantics.

Priority

High

3.4 CR-4 Adaptive Strategy & Build Orders

Description

ProtoBot shall select openings and adapt build orders/tech transitions during the match based on game state and opponent model.

Source

Research goals; competition utility.

Constraints

- Real-time decision making; limited compute; risk of overfitting.

Standards

- Internal policy interfaces between Strategy and Build Managers.

Priority

High

3.5 CR-5 Economy & Production Management

Description

ProtoBot shall manage workers, minerals/gas, supply, and production queues to satisfy strategic goals and sustain unit production.

Source

End-user expectations; research comparability.

Constraints

- Map-dependent saturation and transfer logic; timing windows.

Standards

- BWAPI unit/building command APIs; internal queue formats.

Priority

High

3.6 CR-6 Combat Control (Squad-Level)

Description

ProtoBot shall control combat through squad-level abstractions (grouping units, issuing coordinated orders) to meet the latency budget while remaining tactically effective.

Source

Latency requirement; maintainability goals.

Constraints

- Trade-off between fine micro and frame time; pathfinding costs.

Standards

- Internal squad API; influence-map/path utilities.

Priority

High

3.7 CR-7 Logging, Metrics, & Replays

Description

ProtoBot shall emit structured logs, metrics (e.g., win/loss, frame time, army value, expansions, ladder match results, and rankings), and save replays for analysis and regression testing.

Source

Research reproducibility; competition practice.

Constraints

- I/O overhead must not violate frame time; storage limits.

Standards

- CSV/JSON for metrics; BW replay format; folder conventions `bwapi-data/read|write`.

Priority

High

3.8 CR-8 Configuration & Run Control

Description

ProtoBot shall provide configuration via file and command-line flags (race, map pool, logging level, module toggles) and start/stop hooks for batch runs.

Source

Developer/researcher usability.

Constraints

- Safe defaults; invalid configs must fail fast with clear messages.

Standards

- Consistent key naming; semantic versioning of config schema.

Priority

Moderate

3.9 CR-9 Compatibility & Environment

Description

ProtoBot shall operate with StarCraft: Brood War 1.16.1, BWAPI, fog-of-war enabled, and `setLocalSpeed(0)`; it shall read/write to standard `bwapi-data` folders.

Source

Competition norms; team charter.

Constraints

- Exact game version; map/replay compatibility; OS/runtime differences.

Standards

- BWAPI launcher/paths; standard ladder rules.

Priority

Critical

3.10 CR-10 Robustness & Failure Handling

Description

ProtoBot shall handle common runtime faults (API errors, null units, timeouts) gracefully and exit with descriptive diagnostics; it shall avoid repeated crashes on the same condition within a match.

Source

Usability; demo/competition reliability.

Constraints

- Limited time to recover within frame budget.

Standards

- Internal error codes; minimal on-screen debug in release mode.

Priority

High

3.11 CR-11 Developer Debug Overlays (Optional)

Description

ProtoBot should provide optional on-screen overlays (regions, squad intents, scouting trails) to aid development; overlays shall be disabled in competition builds.

Source

Developer experience.

Constraints

- Overlays must not violate the frame budget when enabled.

Standards

- Toggle via config/flag; consistent color/legend conventions.

Priority

Low

3.12 CR-12 Ethical & Fair Play

Description

ProtoBot shall operate only within allowed APIs and information channels; it shall not exploit hidden game memory, forbidden map knowledge, or non-standard hooks.

Source

Competition rules; academic integrity; instructor expectations.

Constraints

- Some debugging tools may be disallowed in official matches.

Standards

- Competition/ladder rulebooks; team's compliance checklist.

Priority

Critical

3.13 CR-13 Competitive Ladder Compatibility

Description

ProtoBot shall be compatible with automated ladder systems (e.g., BASIL), supporting headless execution, standardized input/output handling, and adherence to competition rules.

Source

Competition rules; BASIL Ladder.

Constraints

- No manual interaction during matches
- Must operate within ladder timing and execution limits

Standards

- BWAPI ladder conventions
- BASIL ladder infrastructure

Priority

High

4 Packaging Requirements

This section defines how ProtoBot will be packaged and delivered to end users. It specifies the distribution channel, release artifacts, directory layout, setup scripts, and accompanying materials so that users can install and run ProtoBot consistently under StarCraft: Brood War 1.16.1 with BWAPI. These requirements focus on the "look and feel" of the deliverable package and avoid duplicating functional or performance requirements defined elsewhere.

4.1 PR-1 Distribution Channel

Description

ProtoBot shall be distributed via a public GitHub repository with tagged releases. Each release shall include a downloadable archive containing binaries and required runtime assets. Releases shall be suitable for deployment in automated ladder environments.

Source

Team charter; instructor guidance; research reproducibility; BASIL ladder.

Constraints

Public availability; institutional policies; repository size limits.

Standards

Semantic Versioning (`MAJOR.MINOR.PATCH`); GitHub Releases.

Priority

Critical

4.2 PR-2 Release Artifacts

Description

Each tagged release shall provide: (a) a Windows `.zip` bundle with compiled binaries, (b) a source archive, and (c) SHA-256 checksums for all downloadable files.

Source

User installability; integrity verification.

Constraints

Windows target (BWAPI); build environment availability.

Standards

SHA-256 checksum file (`SHA256SUMS`); release notes (`CHANGELOG`).

Priority

Critical

4.3 PR-3 Directory Layout

Description

The runtime bundle shall include a top-level directory with the following subfolders and files:

- `bin/` (executables, DLLs), `config/` (default config), `logs/`, `metrics/`, `replays/`
- `scripts/` (launcher batch files), `docs/` (README, QUICKSTART), `licenses/`

Source

Developer usability; competition norms.

Constraints

BWAPI path conventions; Windows filesystem permissions.

Standards

BWAPI folder conventions: `bwapi-data/read/`, `bwapi-data/write/`.

Priority

High

4.4 PR-4 Setup & Launch Scripts

Description

The package shall include scripts to (a) configure paths to StarCraft 1.16.1 and BWAPI, and (b) launch matches at `setLocalSpeed(0)` with fog-of-war enabled. Scripts shall provide sensible defaults and clear error messages.

Source

End-user installability; demo readiness.

Constraints

Windows batch (`.bat`); user permissions; variable install paths.

Standards

Documented environment variables; exit codes for CI.

Priority

High

4.5 PR-5 Configuration Package

Description

A default configuration file (e.g., `config/protobot.ini` or `.yaml`) shall be included with documented keys for race, maps, logging level, module toggles, and output paths. An example file (`config/example.*`) shall be provided.

Source

Research repeatability; user customization.

Constraints

Safe defaults; backward compatibility across minor versions.

Standards

Config schema versioning; key naming conventions.

Priority

High

4.6 PR-6 Documentation Bundle

Description

The release shall include `docs/README` (overview, prerequisites, quick start), `docs/QUICKSTART` (5-minute setup), and `docs/USAGE` (CLI flags, config reference). A link to full online docs (if any) shall be provided.

Source

Instructor expectations; user support.

Constraints

Keep offline docs minimal but complete for first run.

Standards

Markdown documents; consistent section headings.

Priority

High

4.7 PR-7 Compatibility Notes

Description

The package shall include a `docs/COMPATIBILITY` file stating supported OS (Windows 10/11), StarCraft version (1.16.1), BWAPI version, and known limitations.

Source

Competition rules; user expectations.

Constraints

Exact version pinning may be required by BWAPI.

Standards

Explicit version matrix; date-stamped updates.

Priority

High

4.8 PR-8 Logging & Outputs Placement

Description

By default, ProtoBot shall write logs, metrics, and replays to package-local folders (`logs/`, `metrics/`, `replays/`) or to BWAPI standard paths when launched via a ladder. Paths shall be configurable.

Source

Ladder conventions; analysis workflow.

Constraints

I/O throughput; path permissions.

Standards

Filename timestamping; CSV/JSON for metrics; BW replay format.

Priority

Moderate

4.9 PR-9 Licensing & Attributions

Description

The package shall include a `LICENSE` file (e.g., MIT) and `licenses/` folder with third-party notices (BWAPI, libraries). Readme shall include attribution and links.

Source

Open-source compliance; academic integrity.

Constraints

Third-party license terms; competition requirements.

Standards

SPDX identifier in source; NOTICE file.

Priority

Critical

4.10 PR-10 Release Notes & Known Issues

Description

Each release shall provide a `CHANGELOG` summarizing changes and a `KNOWN_ISSUES` file listing major limitations and workarounds (e.g., maps, crashes, timeouts).

Source

User expectations; maintenance practices.

Constraints

Honest disclosure; keep in sync with issues tracker.

Standards

Keep a Changelog format (or equivalent).

Priority

Moderate

4.11 PR-11 Integrity & Provenance

Description

All release archives shall include checksums; optionally, binaries may be Authenticode-signed when feasible. The repo shall tag releases and attach the exact commit hash.

Source

Security; reproducibility.

Constraints

Code signing costs/tooling (optional).

Standards

SHA-256; Git tags and commit IDs.

Priority

Low (signing), High (checksums)

4.12 PR-12 Sample Assets for Verification

Description

The package shall include (a) one sample map reference and (b) a sample replay and metrics file generated by the release, to verify correct installation.

Source

QA; user onboarding.

Constraints

Distribute only permissible assets; small file sizes.

Standards

Replay naming convention; minimal test harness script.

Priority

Low

5 Performance Requirements

This section defines how fast ProtoBot must perform key operations under standard ladder conditions (StarCraft: Brood War 1.16.1, BWAPI, fog-of-war enabled, `setLocalSpeed(0)`). Requirements focus on on-frame latency, startup/shutdown times, data throughput, and resource usage so that end users can run full matches reliably and researchers can reproduce results.

5.1 PRF-1 On-Frame Latency Budget

Description

During match play, ProtoBot shall complete all on-frame processing within ≤ 42 ms for at least 99.95% of frames, with the following hard limits: < 10 frames exceeding 1 s, < 320 frames exceeding 55 ms, and no single frame exceeding 10 s.

Source

Competition norms; team constraints.

Constraints

Hardware variability; OS scheduling jitter.

Standards

BWAPI callback timing; internal timing hooks.

Priority

Critical

5.2 PRF-2 Startup Time

Description

From process launch to first actionable frame, ProtoBot shall initialize within ≤ 5 s on a typical developer machine and ≤ 10 s on minimum-spec hardware.

Source

Demo/competition readiness.

Constraints

Disk I/O, config parsing, map analysis.

Standards

Timing measured wall-clock; one decimal precision.

Priority

High

5.3 PRF-3 Shutdown & Artifact Flush

Description

Upon match end, ProtoBot shall flush logs/metrics and close gracefully within ≤ 3 s (excluding replay save handled by the game client).

Source

Operator usability; CI throughput.

Constraints

File system performance.

Standards

Atomic file writes; final summary in `metrics/`.

Priority

Moderate

5.4 PRF-4 Logging/Metric Throughput

Description

Logging and metric emission shall not cause frame overruns. Sustained logging at up to 2,000 structured events per minute shall add ≤ 2 ms average overhead per frame.

Source

Research reproducibility.

Constraints

I/O bandwidth; JSON/CSV serialization cost.

Standards

Buffered I/O; size-capped rotating logs.

Priority

High

5.5 PRF-5 Memory Footprint

Description

Resident memory shall remain ≤ 600 MB during standard play and shall not grow unbounded over a 60-minute stress match.

Source

Stability; tournament hosts.

Constraints

Map analysis caches; influence maps.

Standards

RSS as reported by OS; leak checks in CI.

Priority

High

5.6 PRF-6 CPU Utilization

Description

Average CPU utilization across a full match shall be $\leq 70\%$ of one physical core, with 95th percentile spikes $\leq 120\%$ of one core (time-sliced).

Source

Host sharing; laptop demos.

Constraints

Single-threaded BWAPI callbacks; limited parallelism.

Standards

OS process stats; median of 10 matches.

Priority

Moderate

5.7 PRF-7 Pathfinding/Combat Budgeting

Description

Combined pathfinding and combat micro shall consume $\leq$ 60% of the per-frame budget on average, with 95th percentile $\leq$ 80%.

Source

Latency headroom for strategy/scouting.

Constraints

Map-dependent chokepoints; large engagements.

Standards

Internal timers per module; rolling window stats.

Priority

High

5.8 PRF-8 Opponent Model Update Rate

Description

Opponent-model updates shall complete within $\leq$ 2 ms per update and run at least every 24 frames ($\approx$ 1 s at fastest speed).

Source

Strategic responsiveness.

Constraints

Partial observability; uncertainty handling.

Standards

Timer-wrapped update function; missed-update counter.

Priority

Moderate

5.9 PRF-9 Config Load & Validation

Description

Loading and validating the configuration shall complete within $\leq$ 200 ms and fail fast with messages $\leq$ 120 characters per error.

Source

Operator UX; batch runs.

Constraints

Schema complexity; file I/O.

Standards

Schema version check; CLI exit codes.

Priority

Low

5.10 PRF-10 Replay & Metrics Size

Description

Per-match artifacts shall not exceed 50 MB replay plus 5 MB metrics/logs under default settings.

Source

Storage management; CI retention.

Constraints

Long matches; verbose logging.

Standards

Compression defaults; rolling retention policy.

Priority

Low

5.11 PRF-11 Frame Overrun Handling

Description

If a frame exceeds 42 ms, ProtoBot shall shed non-critical work next frame and emit a rate-limited warning. Exceeding the hard limits in PRF-1 shall mark the match as a performance failure in the summary.

Source

Operational resilience; diagnostics.

Constraints

Work-shedding must not alter strategic correctness materially.

Standards

Backoff policy; warning counter in metrics.

Priority

High

5.12 PRF-12 Measurement & Reporting

Description

ProtoBot shall record per-module timing (mean, p95, p99), frame time histograms, and total overruns, and include them in the post-match metrics file.

Source

Ablation studies; performance tuning.

Constraints

Measurement overhead ≤ 1 ms average.

Standards

Monotonic clock source; CSV/JSON schema.

Priority

High

6 Safety Requirements

ProtoBot is a software-only AI agent operating within a simulated game environment (StarCraft: Brood War). It does not interface with physical hardware, electrical systems, or mechanical devices, and therefore poses no physical safety risks to users. Safety concerns are instead focused on software safety—ensuring system stability, data integrity, and prevention of unintended or harmful system states (e.g., infinite loops, corrupted files, or unauthorized access).

6.1 SR-1 Software Runtime Safety

Description

ProtoBot shall be designed to avoid unsafe or unstable runtime behavior such as memory leaks, unbounded recursion, or frame stalls that could crash the game client or corrupt user data.

Source

Team quality assurance guidelines; CSE Senior Design lab policy; competition reliability goals.

Constraints

Must operate within the BWAPI API constraints ($\leq$ 42 ms per frame). Must not access unauthorized memory regions of the game client.

Standards

BWAPI runtime conventions; general C++ memory and exception safety best practices.

Priority

Critical

6.2 SR-2 Data and System Integrity Safety

Description

ProtoBot shall not perform file operations that could overwrite or delete non-project data. All read/write actions shall occur only in designated directories (e.g., bwapi-data/write/).

Source

Team charter; instructor policy; standard BWAPI directory structure.

Constraints

Limited file system permissions; user account access rights.

Standards

BWAPI folder conventions; Windows application data access policies.

Priority

High

6.3 SR-3 Ethical and Competition Safety

Description

ProtoBot shall adhere strictly to fair-play and ethical standards. It shall not exploit game memory, inject unauthorized code, or access hidden information unavailable through the official API.

Source

Competition rulebooks (AIIDE, SSCAIT); CSE Senior Design academic integrity policy.

Constraints

All information must be derived from legal BWAPI calls; fog-of-war must remain enabled.

Standards

Competition/ladder guidelines; academic integrity policies.

Priority

Critical

7 Maintenance & Support Requirements

ProtoBot is a software-only AI system designed for autonomous play within StarCraft: Brood War using the BWAPI interface. Maintenance and support focus on ensuring long-term usability, reproducibility of research results, and the ability to debug, extend, and redeploy the bot in future semesters or research projects. No physical maintenance is required. Instead, maintenance activities include codebase updates, bug resolution, configuration management, and compatibility testing with BWAPI and StarCraft versions.

7.1 MSR-1 Source Code Maintenance and Version Control

Description

All ProtoBot source code shall be maintained in a public GitHub repository. The repository shall use semantic versioning and include issue tracking, branches for development and stable releases, and changelogs documenting bug fixes and new features.

Source

Team charter.

Constraints

GitHub free-tier limitations; access control for team members; dependency version changes.

Standards

Semantic Versioning (MAJOR.MINOR.PATCH); Git workflows; Markdown documentation for pull requests.

Priority

Critical

7.2 MSR-2 Configuration and Compatibility Maintenance

Description

ProtoBot shall be maintained to remain compatible with StarCraft: Brood War 1.16.1, BWAPI updates, and Windows 10/11 environments. Configuration files shall include versioned schema headers to ensure backward compatibility.

Source

Competition norms; project charter.

Constraints

BWAPI version changes; external API deprecations.

Standards

BWAPI release notes; internal configuration schema versioning.

Priority

Moderate

7.3 MSR-3 Technical Support and Handoff

Description

Upon project completion, the team shall provide a final repository handoff containing source code, build artifacts, sample logs/replays, and documentation. The handoff shall enable future student teams or researchers to extend ProtoBot without major setup barriers.

Source

Instructor expectations.

Constraints

Limited long-term support after course conclusion; dependency updates outside project scope.

Standards

UTA Senior Design continuation/handoff guidelines; documented environment setup (Visual Studio + BWAPI).

Priority

Moderate

8 Other Requirements

ProtoBot is a modular software system built to operate as a research-grade StarCraft: Brood War AI agent through BWAPI. Beyond functional and performance goals, several architectural, design, and environmental requirements are necessary for long-term reliability, ease of extension, and portability. These "other requirements" ensure that the codebase can be adapted, reused, or built upon in future semesters and that developers can reproduce the working environment consistently.

8.1 OR-1 Modularity and Extensibility

Description

ProtoBot shall maintain a clean modular architecture separating core systems (Information, Scouting, Strategy, Build, Worker, Combat, Request) into independently testable components. Each module shall expose well-defined public interfaces to allow future replacement or improvement without affecting unrelated systems.

Source

Team architecture plan; research extensibility goals.

Constraints

Module interfaces must remain backward-compatible across minor releases.

Standards

Object-oriented C++ design patterns; interface segregation; BWAPI class encapsulation rules.

Priority

High

8.2 OR-2 wCoding Standards and Quality Assurance

Description

All C++ source files shall conform to consistent formatting and naming conventions. Static analysis and compiler warnings (/W4 or -Wall) shall be enabled. Unit tests shall be developed for core utilities such as pathfinding and data parsing.

Source

Team charter; software quality goals.

Constraints

Testing limited to non-interactive modules due to BWAPI event-driven architecture.

Standards

Google C++ Style Guide (adapted); Catch2 or GoogleTest for unit testing.

Priority

High

8.3 OR-3 Security and Data Privacy

Description

ProtoBot shall not transmit, collect, or expose any user data outside local match artifacts. Logs and replays shall remain local to the user's machine unless explicitly uploaded for competition or research.

Source

Ethical and academic policy requirements.

Constraints

BWAPI lacks network sandboxing; relies on local filesystem security.

Standards

BWAPI folder conventions.

Priority

Low

9 Future Items

The following items were listed as low priority, and are unlikely to be completed by the end of the spring 2026 semester due to time constraints.

9.1 CR-11 Developer Debug Overlays (Optional)

Description

ProtoBot should provide optional on-screen overlays (regions, squad intents, scouting trails) to aid development; overlays shall be disabled in competition builds.

Source

Developer experience.

Constraints

- Overlays must not violate the frame budget when enabled.

Standards

- Toggle via config/flag; consistent color/legend conventions.

Priority

Low

9.2 PR-11 Integrity & Provenance

Description

All release archives shall include checksums; optionally, binaries may be Authenticode-signed when feasible. The repo shall tag releases and attach the exact commit hash.

Source

Security; reproducibility.

Constraints

Code signing costs/tooling (optional).

Standards

SHA-256; Git tags and commit IDs.

Priority

Low (signing), High (checksums)

9.3 PR-12 Sample Assets for Verification

Description

The package shall include (a) one sample map reference and (b) a sample replay and metrics file generated by the release, to verify correct installation.

Source

QA; user onboarding.

Constraints

Distribute only permissible assets; small file sizes.

Standards

Replay naming convention; minimal test harness script.

Priority

Low

9.4 OR-3 Security and Data Privacy

Description

ProtoBot shall not transmit, collect, or expose any user data outside local match artifacts. Logs and replays shall remain local to the user's machine unless explicitly uploaded for competition or research.

Source

Ethical and academic policy requirements.

Constraints

BWAPI lacks network sandboxing; relies on local filesystem security.

Standards

BWAPI folder conventions.

Priority

Low

9.5 MSR-4 Documentation and Support Materials

Description

ProtoBot shall include complete documentation for maintainers, including build instructions, configuration guides, and a developer overview of each system module (Protobot Commander, Information, Scouting, Strategy, Build, Worker, Combat, Unit, Request). A troubleshooting section shall describe common runtime issues (missing DLLs, path errors, frame stalls).

Source

Instructor requirements; developer usability goals.

Constraints

Documentation must be versioned alongside code; kept up to date with each tagged release.

Standards

Markdown documentation; README.md;

Priority

Low

9.6 MSR-5 Bug Tracking and Issue Resolution

Description

The development team shall maintain a structured issue tracker (GitHub Issues) for all discovered defects, crashes, or feature requests. Each issue shall include reproduction steps, environment details, and resolution notes.

Source

Team development workflow; reproducibility best practices.

Constraints

Issue resolution dependent on team availability and project phase; restricted access post-course.

Standards

GitHub issue templates; triage labeling conventions (bug, enhancement, documentation).

Priority

Low

References

[1] David Churchhill. Aiide competition rules. https://www.cs.mun.ca/ dchurchill/starcraftaicomp/rules.shtml.

[2] David Churchhill. Bwapi source code and documentation. https://github.com/bwapi/bwapi/.

[3] Loera et al. Protobot starcraft brood war game playing agent project charter. Technical report, The University of Texas at Arlington, 2025.

[1] [2] [3]